

BRIEFINGS

black hat

CLOSED LOOP SECURITY

From Autonomous Exploit to Deployed
Defense in Under 5 Minutes

A Paradigm Shift.

A New Mental Model for Defending in the Age of AI.

This is not about new technology. This is about changing how we use our technology.



Sherwyn Moodley

Founder, Bagheera Labs

Founder of Bagheera Labs, a consultancy specializing in penetration testing for B2B SaaS companies.



Conor Sherman

Global CISO, Sysdig

Global CISO at Sysdig and host of the Zero Signal Podcast



Alessandro Lo Prete

Sr. Threat Detection Engineer, Sysdig

Specializes in cloud security and Linux runtime detection; currently exploring agentic threats and AI in detection workflows.

We have the ingredients. We're still not winning.

The ingredients for a successful AI-driven security program:

Real-time action

Context

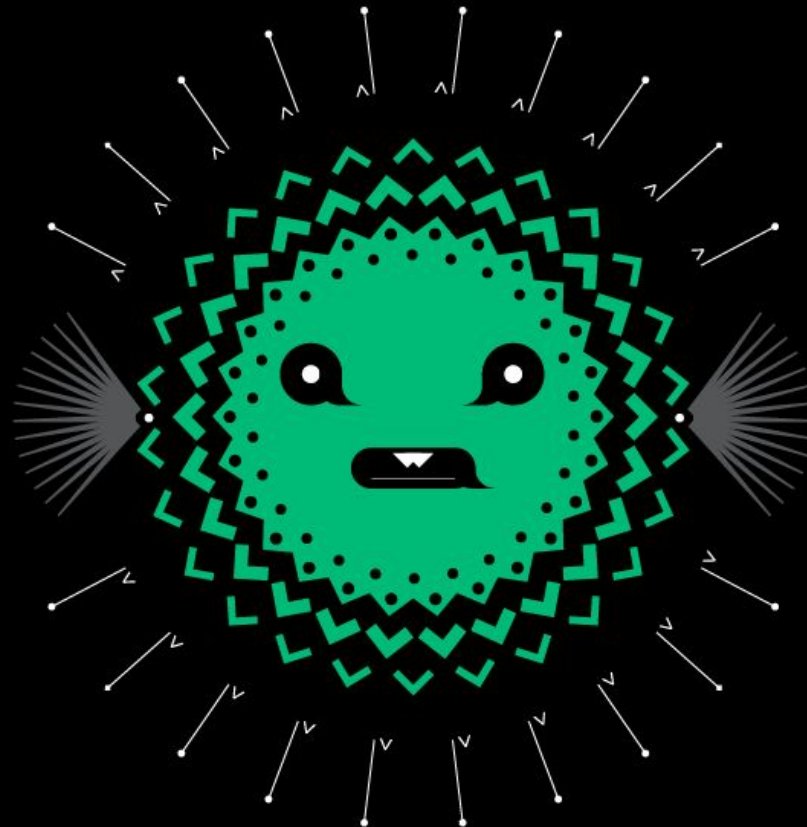
We have an abundance of both.

And yet — **all the evidence says we're still losing.**

THE ADVERSARY IS ALREADY AGENTIC

> JADEPUFFER

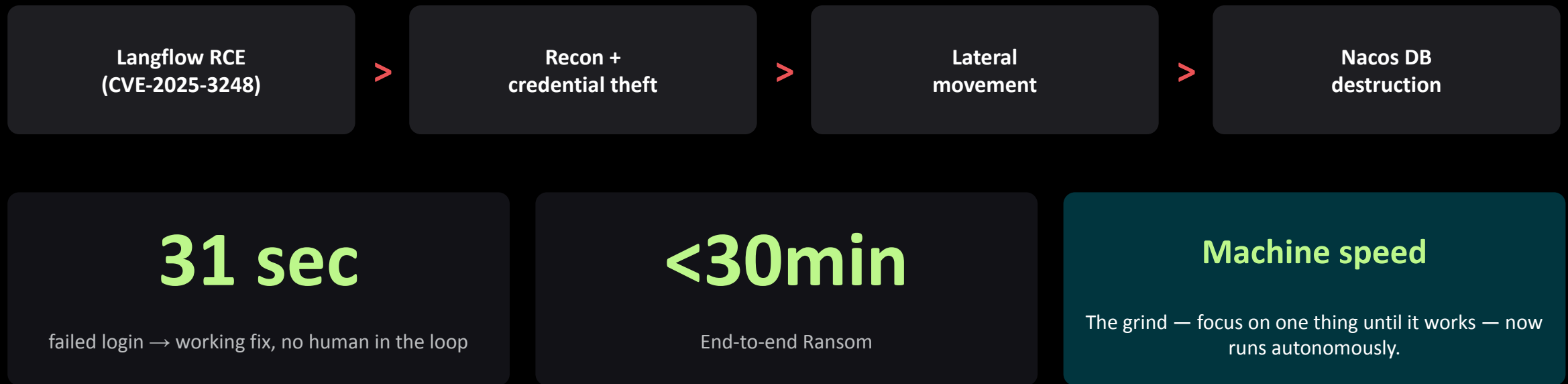
The first observed agentic ransomware operation — uncovered by Sysdig TRT



THE ADVERSARY IS ALREADY AGENTIC

JADEPUFFER

The first documented agentic ransomware operation — an extortion campaign driven end-to-end by an LLM.



Most orgs can't keep pace. What they can do: **real-time detection that takes sub-2-second response the moment an exploit appears.**

Source: Sysdig Threat Research Team — JADEPUFFER, July 1, 2026.

The defender should have every advantage.

THE ADVERSARY

“they need to be lucky once”

Can focus on one thing and keep grinding until they succeed.

THE DEFENDER

“we need to be right all the time”

- Knows the operating environment.
- Understands the context.
- Has home-field advantage.

So why is there such an incredible asymmetry?

Verifier's Law

The ease of training AI to solve a task is proportional to how verifiable the task is. All tasks that are possible to solve and easy to verify will be solved by AI. - Jason Wei,

✗ "Secure all the things"

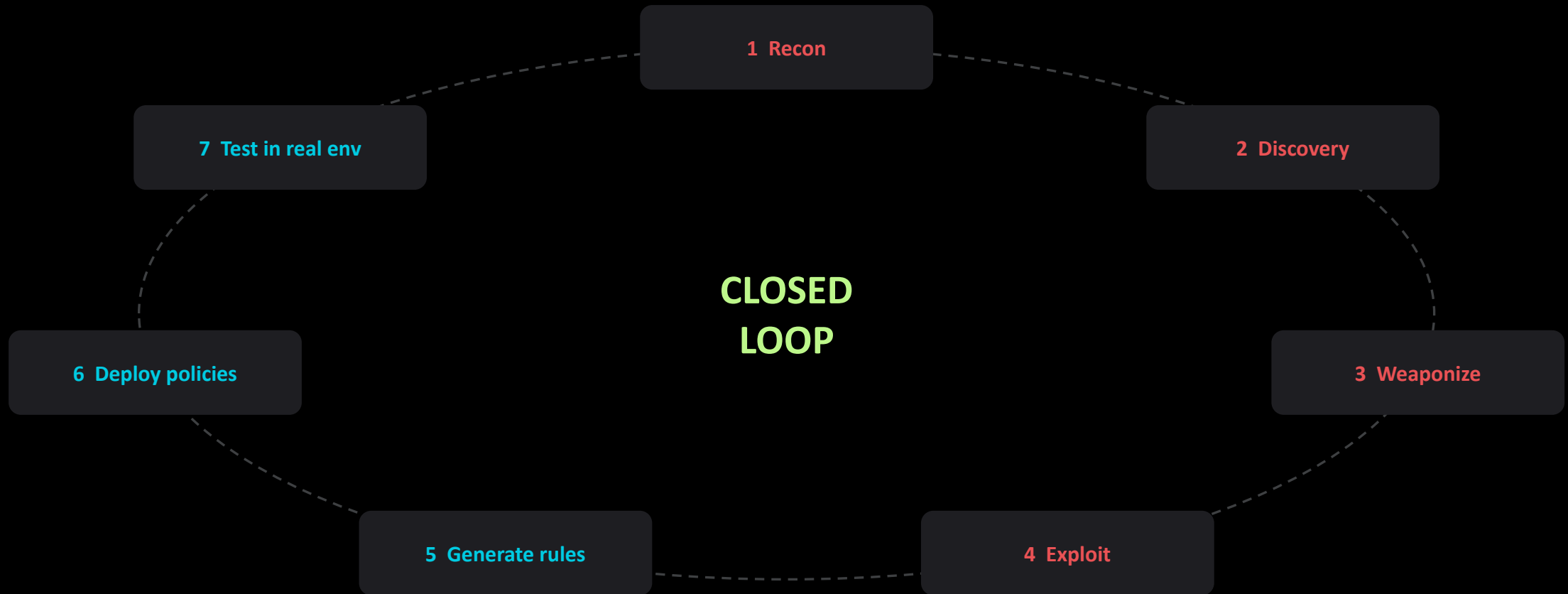
an unverifiable goal

✓ Make the world small

steal unit testing from software engineering

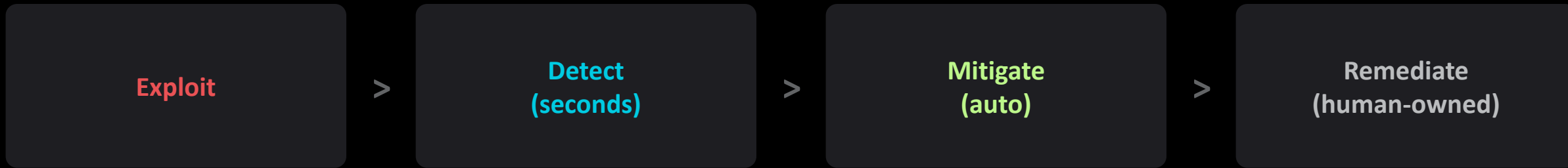
Enter the paradigm of the closed loop.

The closed loop



Attack your own surface → on a proven exploit, write detection + response → re-run to verify → feed it back.

Mitigation buys time for remediation.

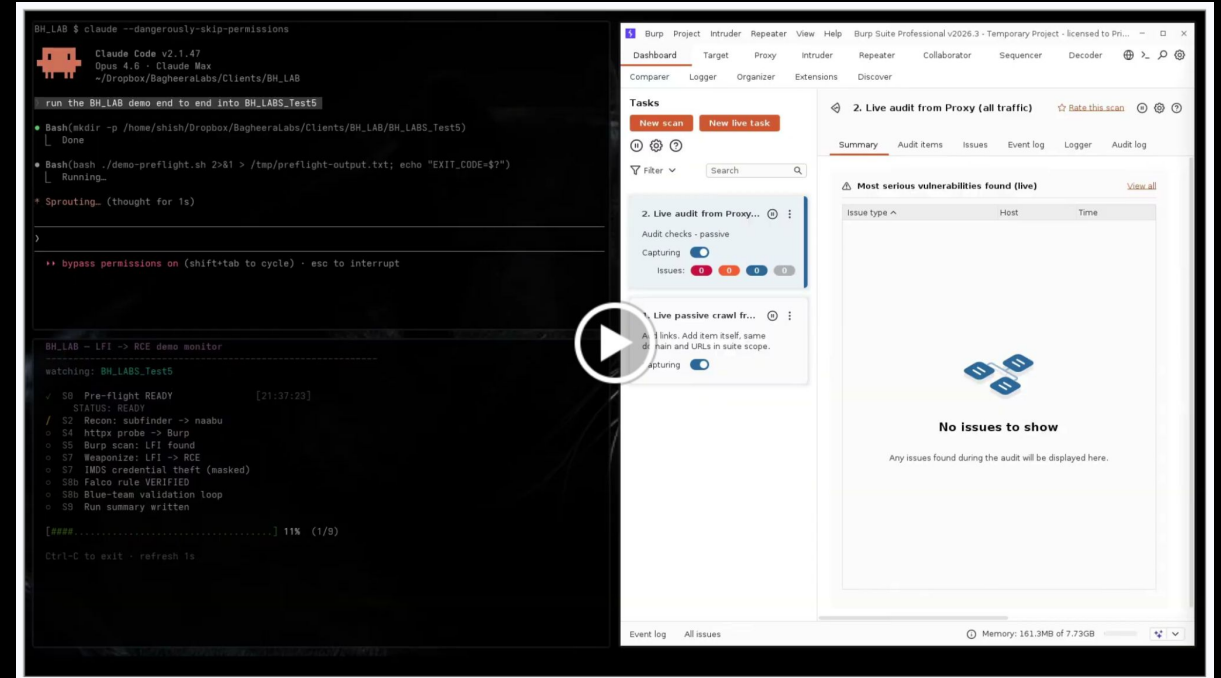


Build real infrastructure for real-time detection and response, and auto-generate a ruleset that fires on exploitation (or near-real-time post-exploit activity). Detect and contain at machine speed — and buy back the time remediation needs.

DEMO

WHAT TO WATCH FOR

- 1 One domain → enumeration → a validated LFI
- 2 Weaponized: LFI → log poisoning → RCE → reverse shell
- 3 The agent writes the Falco rule — re-run the exploit — it's caught



Sherwyn drives live. Backup recording cut to the same beats.

BH_LAB \$ `claude --dangerously-skip-permissions`

 Claude Code v2.1.47
Opus 4.6 · Claude Max
~/Dropbox/BagheeraLabs/Clients/BH_LAB

`run the BH_LAB demo end to end into BH_LABS_Test5`

```
• Bash(mkdir -p /home/shish/Dropbox/BagheeraLabs/Clients/BH_LAB/BH_LABS_Test5)
  | Done
• Bash(bash ./demo-preflight.sh 2>&1 > /tmp/preflight-output.txt; echo "EXIT_CODE=$?")
  | Running...
```

+ Sprouting... (thought for 1s)

>

» `bypass permissions` on (shift+tab to cycle) · esc to interrupt

BH_LAB - LFI -> RCE demo monitor

watching: BH_LABS_Test5

```
✓ S0 Pre-flight READY [21:37:23]
  STATUS: READY
/ S2 Recon: subfinder -> naabu
  ◦ S4 httpx probe -> Burp
  ◦ S5 Burp scan: LFI found
  ◦ S7 Weaponize: LFI -> RCE
  ◦ S7 IMDS credential theft (masked)
  ◦ S8b Falco rule VERIFIED
  ◦ S8b Blue-team validation loop
  ◦ S9 Run summary written
```

[####.....] 11% (1/9)

Ctrl-C to exit · refresh 1s

⚡

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

Organizer

Extensions

Discover

Tasks

New scan

New live task

⏸ ⚙ ?

Filter ▾ Search 🔍

2. Live audit from Proxy... ⏸ ⋮

Audit checks - passive

Capturing ☒

Issues:

0

0

0

0

1. Live passive crawl fr... ⏸ ⋮

Add links. Add item itself, same domain and URLs in suite scope.

Capturing ☒

2. Live audit from Proxy (all traffic) ⭐ [Rate this scan](#) ⏸ ⚙ ?

Summary

Audit items

Issues

Event log

Logger

Audit log

⚠ Most serious vulnerabilities found (live) [View all](#)

Issue type ^	Host	Time
 No issues to show Any issues found during the audit will be displayed here.		

Event log

All issues

ⓘ

Memory: 161.3MB of 7.73GB



Exploitation to mitigation:
under five minutes.

> > >

One chain. Four stages. Each one mirrored.

1 LFI

```
?page=../../var/  
log/auth.log
```

FALCO SEES

php-fpm reads auth.log /
passwd outside web root

>

2 Log poison

```
PHP payload into  
auth.log via ssh user
```

FALCO SEES

<?php written into
auth.log

>

3 RCE

```
?page=...auth.log  
&cmd=id
```

FALCO SEES

php-fpm worker spawns
sh / a recon binary

>

4 Reverse shell

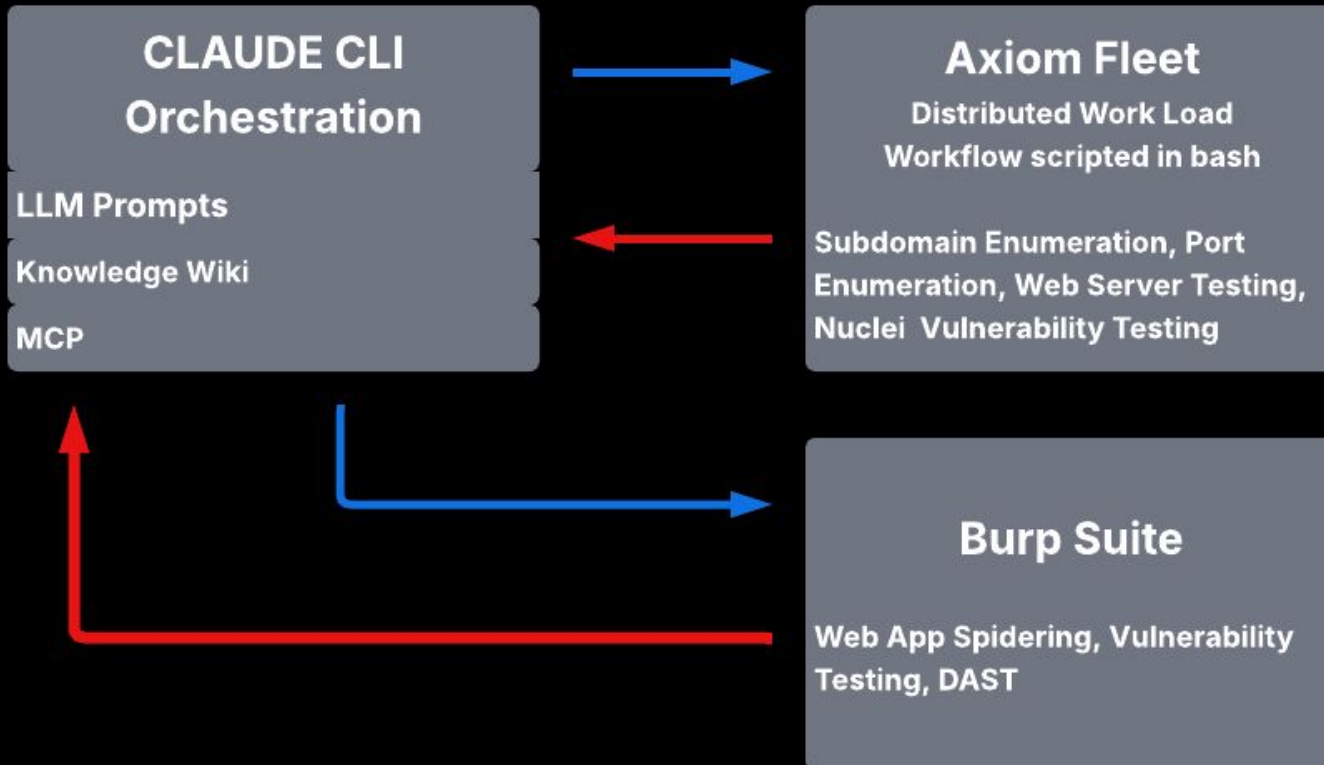
```
python3 -c  
'.../bin/sh -i'
```

FALCO SEES

web-descended python
opens outbound TCP

Each stage also carries a Sigma rule and a Terraform control (not shown).

The Adversarial Brain (Not just an AI Wrapper)



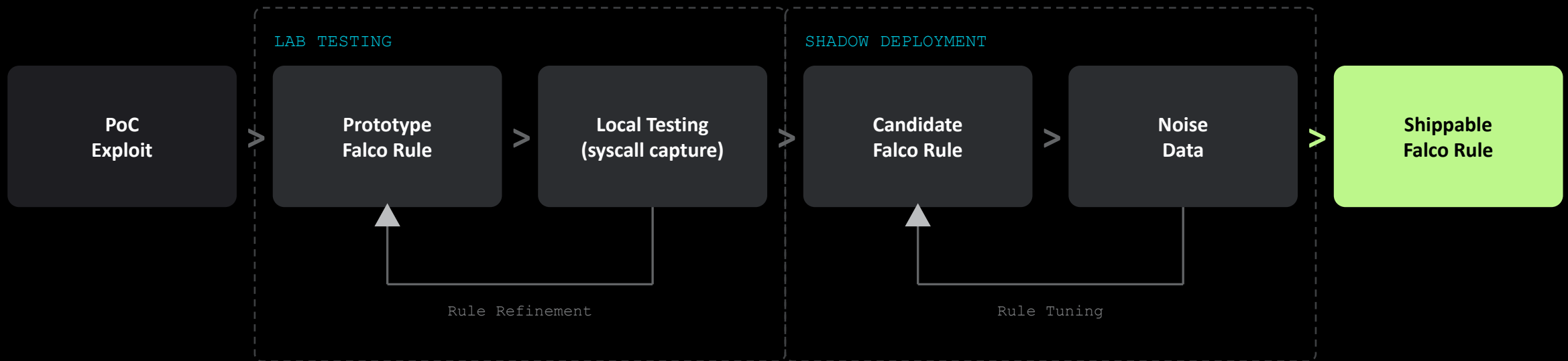
THE BRAIN

- Interprets, correlates, triages, and decides in real time — not glue code over scanners.
- Drives a real distributed toolchain: a recon fleet (subfinder, naabu, httpx, nuclei, katana, ffuf) over MCP, with Burp for deep validation.

THE GROUNDING

- Reasons from a purpose-built research wiki — the playbook it reads before and during every test — not from memory.
- Consistent methodology, real source-level findings, far less hallucination. It cites the playbook; it doesn't invent.

Falco Rule Generation and Validation



Validate twice: refine in the lab until it catches the exploit, then tune in shadow until it's quiet.

Behavioral, not signature.

```
- rule: Shell or Recon Binary Spawned by Web Server Process (RCE)
  desc: >
    Detects a shell or reconnaissance binary spawned as a child of
    Apache/PHP.
    Web workers never spawn interactive shells or run id/whoami/curl in
    normal
    operation — this is the RCE payload from the LFI+log-poisoning chain
    firing.
  condition: >
    evt.type = execve
    and proc.pname in (apache2, php, php-fpm, php8.1, php8.3)
    and proc.name in (bash, sh, dash, id, whoami, uname, curl, wget, nc,
python3, perl,ls, cat)
  output: >
    RCE: web process spawned a shell/recon binary |
    cmd=%proc.name pid=%proc.pid ppid=%proc.ppid parent=%proc.pname
    user=%user.name cmdline=%proc.cmdline
  priority: CRITICAL
  tags: [host, process, mitre_execution, mitre_discovery, T1059.004, T1033]
  enabled: true
```

Web workers never spawn interactive shells in normal operation.

A behavioral signature — not a payload string — so it generalizes across every variant of the exploit.

Detect · Hunt · Mitigate — from one exploit context.

Trigger: only a confirmed_vuln fires the mirror (triage: confirmed / likely / needs-validation / false-positive). A lead is not a finding.

`falco-rule.yaml`

DETECT · RUNTIME

Fires on the behavior — syscalls, process spawns, network.

`sigma-rule.yaml`

DETECT · SIEM

Same behavior in shipped logs — Splunk, Sentinel, Elastic, Loki.

`remediate.tf`

BLOCK + REMEDIATE

Compensating control or the real fix. WAF virtual patch lives inside it.

Why not auto-fix? A wrong detection is noise; a wrong auto-fix is an outage — the downside is asymmetric. Mitigation blocks the exploit without touching the app (reversible virtual patch at the edge). The real code fix stays human-approved.

What exploit once, we *{should}* ~~detect~~ mitigate forever.

We stopped trying to detect everything — and start stacking wins.



One verified test at a time

What we learned building it.

1

Claude rule generation is mediocre; the loop is the win. Ask an LLM for a Falco rule out of the box and it's too broad or too narrow. The value isn't the prompt — it's the iterative candidate-rule loop that tests and refines until it fires cleanly.

2

Automated re-exploitation has a blast radius. The loop proves a rule by re-running the real exploit. Unbounded against a live environment, that itself causes operational damage. Guardrail it: cap iterations, sandbox where possible.

When the world is small enough to verify, it's small enough to automate. When it's small enough to automate, we can start to defend at scale.

We go small. So we can go big.

Q&A

> > >

What are some of the key lessons you've learned?

How should people get started on this?